Genetic Programming Hyper-Heuristics for the Job Shop Scheduling Problem with Interval Durations: Robustness-Aware Interpretable Rules that Generalize Across Instance Sizes

Hernán Díaz

Department of Computing, University of Oviedo, Campus de Gijón, Gijón, 33204, Spain

---

Abstract

Dispatching rules remain the method of choice for scheduling under tight time budgets, but hand-crafted rules leave substantial performance unexploited, and their automated design has scarcely been explored for scheduling problems with non-deterministic processing times. We present a genetic programming (GP) hyper-heuristic for the job shop scheduling problem with interval durations (IJSP), in which durations are known only within an interval and schedules are evaluated by interval arithmetic. The GP evolves priority expressions over an *interval-aware* terminal set that exposes both the worst-case values and the widths of the uncertain quantities, so that reacting to uncertainty is expressible. On the 70 interval Taillard instances, rules from 30 independent evolutions average a relative error of 18.99% ($\pm$1.33) in a single constructive pass, and the best rule attains 17.71%, improving on the strongest constructive baseline (Giffler–Thompson with MWKR tie-breaking, 29.5%) on every instance. They generalize zero-shot across all seven size classes despite being evolved on four instances of one class (20$\times$15),

Email address: `diazhernan@uniovi.es` (Hernán Díaz)

and keep the same margin over the baselines on a second benchmark of classical instances, applied without re-evolution. Evolved rules are compact, human-readable expressions, with the evolutionary hyperparameters configured by irace. A controlled ablation over 30 evolutions per arm delimits the contribution of interval awareness: the width terminals do not improve the expected makespan, but under an objective penalizing interval width they yield substantially narrower predicted intervals and a correspondingly lower deviation of the executed makespan from its prediction, neither attainable without them; a single weight governs the trade-off.
Keywords: interval job shop scheduling, genetic programming,
hyper-heuristics, dispatching rules, scheduling under uncertainty,
robustness

1. Introduction

Real manufacturing environments rarely know the exact duration of an operation in advance. The interval job shop scheduling problem (IJSP) models this uncertainty in a minimal, distribution-free way: each processing time is only assumed to lie within a known interval, and schedules are built and evaluated with interval arithmetic, so that the makespan of a solution is itself an interval and solutions are compared by an interval ranking. Population metaheuristics for the IJSP, among them artificial bee colony variants and tabu search, achieve strong results at the cost of building and evaluating large numbers of complete solutions for every instance [1, 2].

At the opposite end of the computational spectrum are *dispatching rules*: priority functions that build a schedule in a single constructive pass by repeatedly selecting the next operation to start. They combine speed, interpretability and ease of deployment. However, hand-crafted rules (SPT, MOR, MWKR and related rules) leave a large quality gap with respect to search-based methods, as the experiments reported later in this paper confirm.

For *deterministic* scheduling, evolving rules automatically with genetic programming (GP) has been shown to narrow this gap substantially [3, 4]. For scheduling under interval uncertainty, however, the automated design of dispatching rules remains, to the best of our knowledge, unexplored. Moving automated rule design to interval durations raises questions that do not arise in the deterministic case: which attributes a rule should read, when the quantities describing a candidate operation (its duration, its earliest start, the work remaining in its job) are intervals rather than numbers; how candidate operations should be prioritized, and the resulting schedules compared, given that intervals admit no natural total order; and whether the *magnitude* of the uncertainty carries information that is useful for dispatching, or whether it is the position of the intervals that drives the decision. Answering these questions empirically requires exposing uncertainty to the rule as first-class attributes, which is the starting point of this work.

This paper makes the following contributions:

1. A GP hyper-heuristic for the IJSP whose terminal set is *interval-aware*: alongside worst-case processing time, earliest start and work remaining, rules can read the *widths* of these quantities, so that uncertainty-sensitive prioritization is expressible at all (Section 4).
2. An empirical study on the 70 interval Taillard instances showing that evolved rules (i) outperform every deterministic constructive baseline by a wide margin at equal cost, (ii) generalize zero-shot across instance sizes (evolved on 20×15, best class 50×15), and (iii) transfer to a second benchmark of classical instances, keeping the same margin over

the constructive baselines (Section 6).

3. An analysis of *why* the rules work: an anatomy of the evolved expressions (Section 7.1); an ablation that isolates the contribution of the interval-width terminals: they leave the expected makespan unchanged but yield significantly narrower makespan intervals, modestly under a makespan objective and substantially under an objective that rewards predictability (Section 7.2); and an executional-robustness study showing that the evolved rule yields schedules whose realized makespan deviates less from its prediction than the baselines', increasingly so as uncertainty grows (Section 7.3).
4. An automatic configuration study (irace, 200 experiments) of the evolutionary hyperparameters, whose winning configuration is used throughout (Section 5.3).

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 formalizes the IJSP. Section 4 presents the GP hyper-heuristic, from both the hyper-heuristic and the evolutionary viewpoints. Section 5 details the benchmark, the training protocol and the irace configuration study. Section 6 reports the empirical results, and Section 7 analyses them: rule anatomy, the interval-awareness ablation, executional robustness and limitations. Section 8 concludes.

2. Related Work

2.1. Job shop scheduling under interval uncertainty

Interval durations offer a distribution-free uncertainty model requiring only bounds, in contrast to stochastic and fuzzy approaches [5]: they are the natural choice when historical data are insufficient to estimate distributions and a decision maker can only supply a minimum and a maximum duration. Scheduling with interval processing times has been studied across machine environments: single machine [6], flow shop, and flexible job shop [7], among others surveyed in [8]. It has also been studied analytically, through measures of the problem uncertainty induced by the intervals and of the stability of optimal sequences [9].

For the job shop with makespan minimization, which concerns us here, the literature is dominated by population-based metaheuristics. Lei's population-based neighborhood search [10], which ranks interval makespans by a possibility degree, was an early reference point; it was subsequently outperformed by a genetic algorithm with a systematic study of interval rankings [11] and by artificial bee colony variants [1, 2], which report the best published results for this problem. Related objectives have also been addressed under interval uncertainty, notably tardiness minimization [12]. All of these are iterative search methods: they build and evaluate a population of complete solutions and refine it over many iterations for each instance solved. Fast constructive methods for the IJSP have received far less attention; dispatching rules with interval-aware lexicographic comparisons are the de facto baseline.

2.2. Dispatching rules for job shop scheduling

Priority dispatching rules are among the oldest and most widely deployed scheduling heuristics: at every decision point, a priority function ranks the operations that are ready to start and dispatches the best one, building a schedule in a single constructive pass. Classical rules score each ready operation by a single attribute, and we adopt their standard acronyms throughout this paper. *Shortest* and *longest processing time* (SPT, LPT) look at the duration of the operation itself. *Earliest starting time* (EST) prefers the operation that can begin soonest given the current state of its job and its machine. *Most work remaining* (MWKR) and *most operations remaining* (MOR) look instead at the job the operation belongs to, and favor the one with the largest amount of pending work, measured respectively in time units and in number of operations. The *critical ratio* (CR) weighs the work still to be done against the time available until a due date. Surveys catalogue more than a hundred such variants [13, 14]. A step above single-attribute rules, the algorithm of Giffler and Thompson (G&T) [15] restricts each choice to a machine's conflict set, guaranteeing *active* schedules and markedly improving plain rules when a priority criterion is used to break ties inside that set; we write G&T-SPT and G&T-MWKR for the resulting combinations. Stronger constructive procedures exist for the *deterministic* job shop, notably the shifting bottleneck heuristic [16], which repeatedly solves one-machine subproblems with heads and tails by means of Carlier's algorithm [17]. These fall outside the scope of the present work: their machinery rests on exact one-machine optimization and critical-path arguments that do not extend directly to interval arithmetic, where no natural total order is available. Two lessons emerge from this literature: rules are fast, transparent and easy to deploy, but hand-crafting them plateaued, as no fixed combination of attributes performs well across shop configurations and objectives [14]. That plateau is what motivated the *automated* design of rules.

2.3. Automated design of dispatching rules

Genetic programming [18, 19] evolves a population of expressions, usually represented as trees, by applying selection and variation operators directly on their structure. GP hyper-heuristics apply this machinery to *priority ex-*

pressions over scheduling attributes, and constitute the standard approach to automated rule design for production scheduling [3, 4]. Evolved rules have been shown to outperform hand-crafted ones across job shop variants, with active research on surrogate fitness, feature selection, and multi-objective formulations. Beyond single rules, a further line combines several evolved rules into *ensembles* that jointly construct or vote on a solution [20], and recent work explores alternatives to evolutionary search for the same design task, namely systematic tree search over the space of expression trees [21], reporting rules of comparable quality but smaller size and better readability. All of this work targets deterministic (or dynamic-stochastic) settings. Closest to our setting, Karunakaran et al. [22] evolve rules for dynamic job shops with uncertain processing times, exposing summary statistics of observed deviations (an exponential moving average) to the rule; uncertainty there is realized stochastically during simulation, and rules are evaluated on sampled scenarios. A recent survey of learned optimization for the job shop [23], covering both GP and DRL constructive approaches, records no work on interval-valued processing times. To the best of our knowledge, evolving rules whose *inputs* are interval attributes, and whose fitness is computed by interval arithmetic rather than by sampling realizations, has not been explored before.

Finally, deep reinforcement learning provides an alternative family of learned constructive methods for the JSP [24, 25]; GP rules and DRL policies are increasingly studied as competing learning paradigms for the same construction task, and controlled comparisons between them under common protocols have been identified as a gap in the recent literature [23]. Both families occupy the same computational class at deployment: a single constructive pass.

3. The Interval Job Shop Scheduling Problem

An IJSP instance consists of n jobs and m machines. Job j is a chain of m operations $o_{j1} \prec o_{j2} \prec \cdots \prec o_{jm}$, each requiring a distinct machine $\mu(o_{jl})$ exclusively and non-preemptively. The duration of operation o is an interval $\mathbf{p}_o = [\underline{p}_o, \overline{p}_o]$ with $0 < \underline{p}_o \leq \overline{p}_o$: the only assumption is that the realized duration lies within these bounds. As in this definition, boldface denotes an interval-valued quantity throughout, and the underlined and overlined symbols its lower and upper bounds.

Schedule construction uses interval arithmetic. For intervals $\mathbf{a} = [\underline{a}, \overline{a}]$ and $\mathbf{b} = [\underline{b}, \overline{b}]$, the two operations required by makespan minimization are

$$\mathbf{a} + \mathbf{b} = [\underline{a} + \underline{b},\, \overline{a} + \overline{b}], \tag{1}$$

$$\max(\mathbf{a}, \mathbf{b}) = [\max(\underline{a}, \underline{b}),\, \max(\overline{a}, \overline{b})], \tag{2}$$

both component-wise [11]. A solution determines a task processing order π (a permutation with repetition of jobs, decoded left to right). Following the notation of [11, 2], let $\mathrm{PM}_o(\pi)$ and PJ_o denote the predecessors of task o in its machine (according to π) and in its job. The *semi-active* schedule [26] starts each task at

$$\mathbf{s}_o(\pi) = \max\left(\mathbf{c}_{\mathrm{PJ}_o}(\pi),\, \mathbf{c}_{\mathrm{PM}_o(\pi)}(\pi)\right), \qquad \mathbf{c}_o(\pi) = \mathbf{s}_o(\pi) + \mathbf{p}_o, \tag{3}$$

so starting and completion times of all tasks are themselves intervals. The problem can be stated as

$$\min_R \mathbf{C}_{\max} \tag{4}$$

$$\text{s.t.} \quad \mathbf{C}_{\max} = \max_{1 \leq j \leq n} \{\mathbf{c}_{o_{jm}}\}, \tag{5}$$

$$\underline{s}_{o_{jl}} \geq \underline{c}_{o_{j,l-1}}, \quad \overline{s}_{o_{jl}} \geq \overline{c}_{o_{j,l-1}}, \qquad 2 \leq l \leq m,\ 1 \leq j \leq n, \tag{6}$$

$$\underline{s}_o \geq \underline{c}_{o'} \vee \underline{s}_{o'} \geq \underline{c}_o, \quad \overline{s}_o \geq \overline{c}_{o'} \vee \overline{s}_{o'} \geq \overline{c}_o, \quad \forall o \neq o' : \mu(o) = \mu(o'), \tag{7}$$

where Eq. (5) defines the interval makespan as the component-wise maximum of the job completion times, Eq. (6) enforces precedence within each job and Eq. (7) forbids overlaps on each machine, in both interval components [11, 2]. The minimum in (4) is taken with respect to an interval ranking R, since intervals admit no natural total order. We rank schedules lexicographically, comparing upper bounds first and using the lower bound only to break ties:

$$\mathbf{a} \preceq \mathbf{b} \iff \overline{a} < \overline{b} \ \vee \ \left(\overline{a} = \overline{b} \wedge \underline{a} \leq \underline{b}\right), \tag{8}$$

one of the standard rankings of the IJSP literature and the one found in [11] to yield the most robust schedules. All methods in this paper, the GP rules and every baseline, share one implementation of this decoder and evaluator, so reported differences cannot stem from evaluator discrepancies.

Performance is reported as the relative error of the expected makespan with respect to a crisp reference bound LB of each instance,

$$\mathrm{RE} = \frac{E[\mathbf{C}_{\max}] - \mathrm{LB}}{\mathrm{LB}} \times 100, \qquad E[\mathbf{C}_{\max}] = \frac{\underline{C}_{\max} + \overline{C}_{\max}}{2}, \tag{9}$$

where $E[\mathbf{C}_{\max}]$ is the expected value of the interval makespan under the uniform distribution on $[\underline{C}_{\max}, \overline{C}_{\max}]$, and LB is a bound of the underlying deterministic instance: Taillard's bounds for the Taillard-derived instances, the best known ones [1] for the classical set. It serves as an indicative common reference, not as a bound for the interval problem, for which none is available.

A schedule is eventually *executed* under one realization of the durations, and a second quantity of interest is how well its a-priori interval predicts that execution. Following [11], the $\bar{\varepsilon}$ robustness of a schedule with a fixed processing order is the average relative deviation of its executed makespan from the predicted expected value, over K realizations of the durations:

$$\bar{\varepsilon} = \frac{1}{K} \sum_{k=1}^{K} \frac{\left|C_{\max}^{\mathrm{ex},k} - E[\mathbf{C}_{\max}]\right|}{E[\mathbf{C}_{\max}]}, \tag{10}$$

Table 1: Interval-aware terminal set, with exact definitions. For an eligible operation o of job j on machine $\mu(o)$: $\mathbf{p}_o = [\underline{p}_o, \overline{p}_o]$ is its duration interval; $\mathbf{C}_j$ and $\mathbf{C}_{\mu(o)}$ are the interval completion times of the last scheduled operation of job j and of machine $\mu(o)$; $\mathcal{R}_o$ is the set of operations of j after o; $\mathcal{E}$ is the current eligible set. Terminals are evaluated on *raw* (unnormalized) values.

Terminal	Definition	Role
PT	$\overline{p}_o$	worst-case processing time
PTW	$\overline{p}_o - \underline{p}_o$	duration uncertainty
EST	$\overline{e}_o = \max(\overline{C}_j, \overline{C}_{\mu(o)})$	worst-case earliest start
ESTW	$\overline{e}_o - \underline{e}_o$, with $\underline{e}_o = \max(\underline{C}_j, \underline{C}_{\mu(o)})$	start uncertainty
WKR	$\sum_{q \in \mathcal{R}_o} \overline{p}_q$	worst-case work remaining
WKRW	$\sum_{q \in \mathcal{R}_o} (\overline{p}_q - \underline{p}_q)$	remaining-work uncertainty
NOR	$\lvert \mathcal{R}_o \rvert$	operations remaining
SLACK	$\overline{e}_o - \min_{o' \in \mathcal{E}} \overline{e}_{o'}$	start slack within eligible set
ONE	1	constant

where $C_{\max}^{\mathrm{ex},k}$ is the crisp makespan obtained by executing that same order when the durations turn out to take the values of realization k. A low $\bar{\varepsilon}$ therefore means that what was promised before execution is close to what is obtained after it.

4. GP Hyper-Heuristic for the IJSP

We describe the method from two complementary viewpoints: as a *hyper-heuristic*, that is, what an individual is and how it turns an IJSP instance into a schedule (Section 4.1); and as an *evolutionary algorithm*, that is, how the population of rules is initialized, varied and selected (Section 4.2).

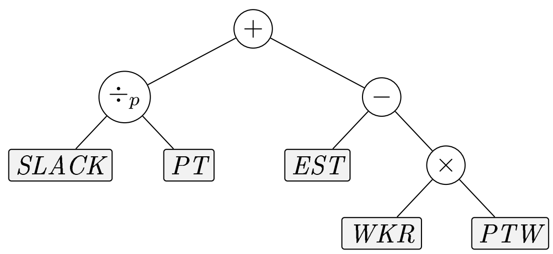

Figure 1: An individual is a priority expression tree; leaves are interval-aware attributes of the candidate operation (here the illustrative rule $SLACK/PT + EST - WKR \cdot PTW$). At each decision point the tree is evaluated on every eligible operation and the lowest-priority one is dispatched.

4.1. Hyper-heuristic view: from expression trees to schedules

An individual is a priority expression tree (Figure 1). Leaves are drawn from the interval-aware terminal set of Table 1, and internal nodes from the function set

$$\mathcal{F} = \{\, +,\ -,\ \times,\ \div_p,\ \min,\ \max,\ \mathrm{neg}\,\}, \tag{11}$$

where neg is the unary sign change and $\div_p$ denotes protected division, defined as $a \div_p b = a/b$ if $|b| > 10^{-9}$ and a otherwise. The rule is deployed inside a semi-active schedule generation scheme (SGS): starting from the empty schedule, at each of the $n\,m$ decision points the set $\mathcal{E}$ of eligible operations (those whose job predecessor is already scheduled) is formed, the tree is evaluated on the attributes of every $o \in \mathcal{E}$, and the operation with *lowest* priority value is appended at its earliest feasible start on its machine, Eq. (3).

Because the lowest value is dispatched, a criterion that seeks to *maximize* an attribute must be expressed through the unary neg operator. Classic rules are thus points of this search space: shortest processing time is the single-node tree PT, whereas most work remaining is $\mathrm{neg}(WKR)$. We inject both as seeded rules in the initial population.

Priority ties at dispatch time are broken by eligible set order (lowest job index first), which is deterministic; hence a rule maps each instance to exactly one schedule.

4.2. Evolutionary components

The initial population is built with the *ramped half-and-half* method, introduced by Koza [18] and still the standard choice for tree-based GP [19], which combines two tree-generation procedures over a range of depths. The *full* method expands every branch with function nodes until the prescribed maximum depth is reached, placing terminals only at that level, and thus produces complete trees whose leaves all lie at the same depth, whereas the *grow* method may close a branch earlier by placing a terminal, with probability 0.3 at each node in our implementation, and thus produces trees of irregular shape and variable size. In both cases the function at an internal node is drawn uniformly from $\mathcal{F}$ and a leaf uniformly from the terminal set, the arity of the chosen function determining the number of branches to expand. Each individual draws its maximum depth uniformly from $\{2, 3, 4\}$ and is generated by either method with equal probability. The last two members of the population are not generated at random: they are the two seeded rules of Section 4.1, inserted directly. Every individual is evaluated by the mean RE of its deterministic constructive pass over the four training instances, computed with the same environment, decoder and evaluator used by all baselines.

Each generation then proceeds as follows. Parents are chosen by tournament, drawing k individuals uniformly without replacement from the current population and returning the fittest of them. With probability p_c two parents are recombined by *subtree crossover* [18, 19], the standard recombination op-

Algorithm 1 GP hyper-heuristic for the IJSP

$P \leftarrow$ ramped half-and-half$(pop - 2) \cup \{$SPT, MWKR$\}$
evaluate(P) ▷ mean RE of one constructive pass on the training instances
for $g = 1$ **to** *gens* **do**
  $P' \leftarrow$ elite$(P,\ e)$ ▷ elitism
  while $|P'| < pop$ **do**
    if rand() $< p_c$ **then**
      $p_1, p_2 \leftarrow$ tournament(P, k), tournament(P, k)
      $child \leftarrow$ subtreeCrossover$(p_1,\ p_2)$
    else
      $child \leftarrow$ subtreeMutation(tournament(P, k))
    end if
    $P' \leftarrow P' \cup \{child\}$ ▷ fitness ∞ if size $> c$
  end while
  $P \leftarrow$ evaluate(P')
end for
return best rule in P

erator for tree-based GP. The offspring is a copy of the first parent in which one subtree has been replaced by a subtree taken from the second: a node is drawn uniformly at random in each parent, leaves included, and the subtree rooted at the node chosen in the first parent is substituted by the subtree rooted at the node chosen in the second. In its classical formulation the operator exchanges both subtrees and returns two offspring; we keep a single one per recombination. With the complementary probability $1 - p_c$ a single parent undergoes *subtree mutation* [18, 19]: a node is again drawn uniformly

Table 2: Evolution parameters: the configuration selected by irace (Section 5.3), used for every experiment reported in this paper.

Population / generations	100 / 50
Initialization	ramped half-and-half (depths 2–4)
Seeded individuals	PT and neg(WKR)
Selection	tournament, size $k = 7$
Crossover / mutation prob.	$p_c = 0.77$ / 0.23 (subtree, depth ≤ 3)
Elitism	$e = 2$
Tree-size cap	$c = 30$ nodes (violations: fitness ∞)
Fitness	mean RE, deterministic rollout, TA11–TA14
Seeds	1–30 (RNG of evolution and environment)

at random, and the subtree rooted at it is discarded and replaced by a freshly generated random tree, built with the grow method and maximum depth 3 over the full primitive set. Offspring whose size exceeds a cap of c nodes receive infinite fitness and are thereby excluded from selection. Note that the depth bounds above apply only to the trees created from scratch, at initialization and inside mutation: the depth of an individual is never constrained during the run, so recombination routinely produces expressions deeper than those of the initial population, the number of nodes being the only quantity that is bounded.

Replacement is fully generational, the offspring becoming the next population, except for the e best individuals of the current generation, which are copied into it unchanged. The process stops after a fixed number of generations and returns the best rule found. Algorithm 1 summarizes the generational loop and Table 2 lists the parameter values.

4.3. Randomized multi-start extension

A deterministic rule produces one schedule. For matched comparisons with sampling-based methods we wrap the evolved rule in GRASP-style randomization: with probability ε=0.1 the dispatched operation is drawn uniformly from the eligible set, otherwise the rule decides; sample 0 remains deterministic. Best-of-N then reports the lexicographically best schedule. This uniform perturbation of a single rule is the simplest member of a family of randomized rule-based builders; an alternative swaps among a *set* of evolved rules during construction [27].

5. Experimental Setup

5.1. Benchmark instances

We use two benchmarks, both taken from the IJSP literature [11, 1, 2]: the 70 interval versions of Taillard's instances (TA1–TA70, seven size classes from 15×15 to 50×20), and the 12 classical instances on which the published IJSP metaheuristics report results (FT10, FT20, six instances of the La family and ABZ7–9, of sizes 10×10 to 20×15). Both were obtained from their crisp counterparts with the generation scheme introduced in [11]: for every operation a value δ is drawn uniformly at random in $[0,\, 0.15\,p]$, where p is its original duration, and that duration becomes the integer interval $[p-\delta,\, p+\delta]$, machine routes being unchanged. The interval is symmetric around p, so the crisp instance is exactly the midpoint scenario of its interval version. Since rules are evolved on Taillard instances only and applied to the classical set without any re-evolution, the latter doubles as a cross-family generalization test (Section 6.4). Both sets are released with this paper (see Data availability).

5.2. Training protocol, data split and baselines

Rules are evolved on TA11–TA14 (20×15), and TA15–TA20 serve as development set: they never contribute to fitness, but design and configuration decisions (including the irace study of Section 5.3) were selected on their performance, so we flag them as subject to development reuse. The remaining 60 Taillard instances and the 12 classical instances are fully unseen by every design decision and constitute the generalization test.

Each evolution runs a population of 100 rules for 50 generations and returns the best rule of its final population. Thirty independent evolutions, differing only in their random seed, therefore yield thirty rules; these are evaluated on the 70 benchmark instances, and the one with the lowest mean RE is the *featured rule*.

5.3. Hyperparameter configuration with irace

Four parameters govern the behavior of the evolution: tournament size, crossover probability, tree-size cap and elitism. We configured them with irace over a budget of 200 experiments, each one a full evolution (population 100, 50 generations) on the training instances, scored by the resulting rule's RE on the development set. Table 3 lists the parameter space and the winning configuration, which is the one used for every experiment reported in this paper (Table 2). The elite configurations agree on a high selection pressure (tournament size 7) and a crossover probability in the range 0.72–0.77; the tree-size cap does not discriminate, with surviving elites spanning caps of 20, 30 and 40.

Table 3: Hyperparameter space explored by irace (200 experiments; cost = development-set RE of the evolved rule) and the winning configuration.

Parameter	Range / values	irace winner
Tournament size	$\{2, 3, 4, 5, 7\}$	7
Crossover prob. p_c	[0.5, 0.95]	0.77
Tree-size cap	$\{20, 30, 40, 60\}$	30
Elitism	$\{1, 2, 4\}$	2

6. Results

6.1. Evolution behavior and seed stability

Over 30 independent evolutions, training RE converges to 14.9 ± 1.5 (range 12.4–19.7), with most of the improvement realized by generation 25–30 (Figure 2). The evolved rules combine the classical dispatching ingredients in recognizable form. Figure 3 shows the featured rule, obtained with seed 1: 26 nodes over 12 leaves. Every terminal of the tree is non-negative by construction and processing times are at least one time unit, so both guards it contains resolve to the same branch at every decision. On the problem domain the tree is therefore exactly equivalent to the expression

$$SLACK^2 \;+\; 2\,PT \;-\; WKR \;-\; WKRW \;-\; 1. \tag{12}$$

We verified this identity at each of the 37,250 dispatching decisions the rule takes on the benchmark (1.29 million candidate evaluations). Since the lowest value is dispatched, the rule reads directly: $-WKR$ is most-work-remaining, $2\,PT$ is shortest-processing-time at double weight, $SLACK^2$ penalizes quadratically any operation that cannot start at the earliest available time, and the constant is immaterial to the ranking. The remaining term,

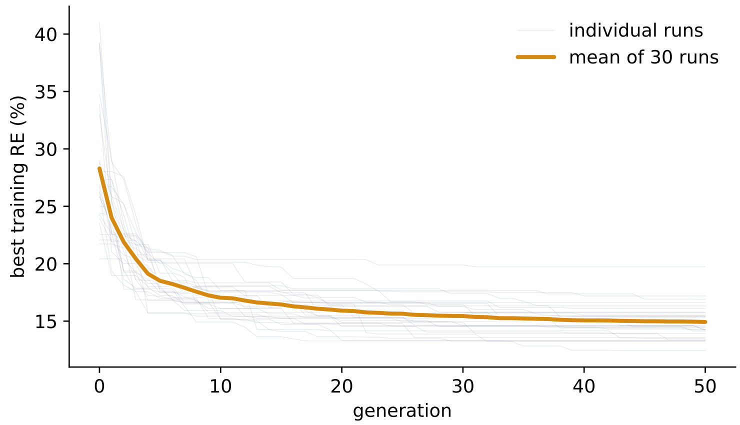

Figure 2: Best training fitness per generation over the 30 independent evolutions: each individual run and their mean.

$-WKRW$, is the interval-aware one: among jobs with comparable remaining work the rule advances those whose remaining work is *more uncertain*, resolving that uncertainty early rather than carrying it to the end of the schedule. The evolved rule is thus a weighted blend of MWKR and SPT, regularized by a quadratic earliest-start term and tilted by the uncertainty of the work ahead.

6.2. Generalization to the full benchmark

Table 4 reports single-rollout RE per size class over all 70 instances, as mean $\pm$ standard deviation across the 30 evolved rules and for the single best rule. The best rule attains 17.71% global mean. Evolved only on 20×15, its best class is the unseen 50×15 (13.8%), and the same pattern holds on average (13.7 ± 1.5). Because all terminals are per-operation attributes with no size-bound quantities, transfer requires no retraining or rescaling.

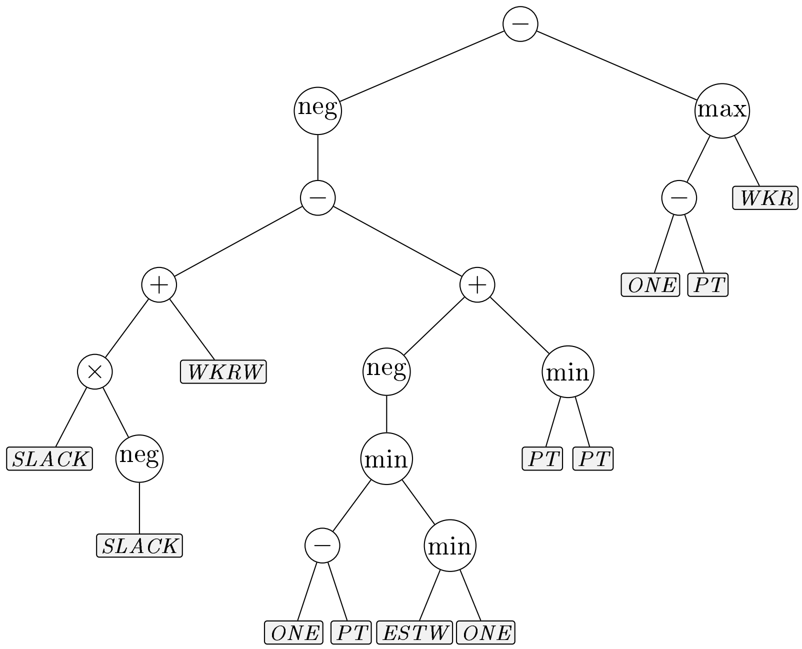

Figure 3: The featured rule (seed 1), which attains 17.71% RE over the 70 instances. Circles are functions and shaded boxes the interval-aware terminals of Table 1; the operation with the lowest value of the expression is dispatched. The tree is shown exactly as evolved; Eq. (12) gives its simplified form.

Table 4: RE (%) per size class, single deterministic rollout, over all 70 Taillard interval instances: mean ± sd across the 30 independently evolved rules, and the best evolved rule.

	15×15	20×15	20×20	30×15	30×20	50×15	50×20	All
Mean of 30	18.1±1.3	18.0±1.5	20.9±1.7	21.4±1.7	25.7±1.9	13.7±1.5	15.1±1.5	18.99±1.33
Best rule	15.7	16.6	18.1	21.1	24.1	**13.8**	14.6	**17.71**

6.3. Position among constructive methods

Table 5 reports every constructive baseline under the protocol of this paper, one deterministic pass per instance with the same decoder and evaluator for all methods, together with a uniformly random dispatcher as a reference point. Since these rules are deterministic, the standard deviation is taken across instances and measures how uniform a method is over the benchmark rather than any run-to-run variability; the random dispatcher, the one stochastic entry, is averaged over 10 dispatches per instance. Times are wall-clock averages over the same 70 instances with 5 repetitions each; being implementation-bound, they are informative only in comparison between rows. The two GP entries are the mean over the 30 evolved rules and the best of them, the mean being what a single evolution should be expected to yield.

Two observations are needed to read the table correctly. First, the plain rules that ignore machine availability (SPT, LPT and the critical ratio CR, the latter being a due-date rule applied here without due dates) perform *worse than random* in this setting. This is a property of the schedule generation scheme, not a defect of the criteria: because a solution is decoded as a permutation whose operations are appended to their machines, systematically dispatching by processing time alone produces highly unbalanced permutations. The same criterion embedded in the Giffler–Thompson conflict set, which restores classical dispatching semantics, behaves as expected (SPT: 627% as a plain rule versus 70.6% under G&T). Second, the criteria that do account for the state of the shop (EST, MOR, MWKR) are clearly better than random, and the strongest baseline overall is G&T-MWKR at 29.5%, which is the reference we use throughout.

Against that strongest baseline, the evolved rules average 18.99 ± 1.33 RE over the 30 evolutions, and the best rule attains 17.71%.

The gap holds at the level of individual instances: the best evolved rule outperforms each of the eight deterministic baselines on every one of the 70 instances, and a paired Wilcoxon signed-rank test rejects equality against each of them at $p < 10^{-6}$ ($z = -7.3$). The narrowest margin over the whole

Table 5: Constructive baselines on the 70 interval Taillard instances: mean RE (%), standard deviation across instances, and mean time in seconds of one constructive pass.

Method	RE (%)	sd	Time (s)
LPT	703.6	168.0	0.23
SPT	627.2	152.2	0.23
CR	625.7	140.6	0.26
Random dispatcher	*127.2*	*13.9*	*0.09*
G&T-SPT	70.6	10.0	0.29
MWKR	64.7	10.4	0.33
MOR	45.5	10.1	0.33
EST	42.3	10.6	0.28
G&T-MWKR	**29.5**	6.4	0.38
GP rule (mean of 30)	18.99	4.96	0.41
GP rule (best of 30)	**17.71**	5.23	0.34

comparison is 3.9 points, against G&T-MWKR on a single 20×15 instance; against the best plain rule (EST) the smallest margin is 6.3 points. Per-instance figures are given in Table A.9 (appendix). This quality comes at no evaluation penalty over hand-crafted rules: evaluating the evolved expression tree over the eligible set costs what evaluating a hand-crafted criterion costs. Averaged over the 70 instances, a complete constructive pass takes 0.34 s with the evolved rule against 0.33 s with MOR and 0.38 s with G&T-MWKR, a spread of 14% between the three; all of them reduce to one vectorized pass over the eligible operations per decision, so the choice of priority expression contributes little to the cost of a pass. Both cost and quality thus place the evolved rule with the constructive methods, while closing much of the quality gap that separated them from search-based methods; that comparison is deferred to Section 6.4, where published results are available on a common benchmark.

6.4. Cross-family generalization: the 12 classical instances

The 12 classical instances serve a double purpose: they test the evolved rule on instance families and sizes it has never seen, since it was evolved on four interval Taillard 20×15 instances and is applied here without any re-evolution, and they enable a direct comparison with the *published* results of the IJSP metaheuristics on their own benchmark and metric (Table 6). Those methods belong to different algorithmic families, evolutionary computation and swarm intelligence: a genetic algorithm evolving a population of permutations [11] and three artificial bee colony variants [1, 2], each of which runs a separate search on every instance. Alongside the deterministic pass, Table 6 reports the randomized extension of Section 4.3 at best-of-64 and best-of-1024: N independent ε-randomized passes of the same rule, of which the best schedule is kept, with no refinement or recombination among samples.

The two families also differ in how they spend computation: best-of-1024 evaluates 1024 mutually independent samples per instance, whereas the metaheuristics evolve a population of 250 individuals [11, 1] through successive generations, each of which depends on the previous one. In wall-clock terms, a single constructive pass over these instances takes 0.03–0.13 s in our Python implementation, 0.07 s on average, against the 0.5–2.2 s reported for the genetic algorithm and the 1.9–6.8 s reported for fEABC on the same instances [1], both implemented in C++. Best-of-1024, however, requires 37–134 s of sequential Python time depending on instance size, and is therefore *not* faster than the published metaheuristics.

Table 6: RE (%) on the 12 classical interval instances: our methods (deterministic single pass and GP-ε best-of-N; rule evolved on interval Taillard 20×15, applied zero-shot) versus the published average results (30 runs per instance) of the IJSP metaheuristics GA [11], ABC$_{E3}$, fEABC [1] and ESABC [2]. LB is the best-known lower bound of the expected makespan.

		Ours (zero-shot)			Published (avg of 30)			
Inst.	LB	GP	GP-ε_{64}	GP-ε_{1024}	GA	ABC$_{E3}$	fEABC	ESABC
FT10	930	20.4	8.4	6.6	5.2	4.1	3.5	3.0
FT20	1165	13.2	12.1	10.8	4.4	1.7	1.8	1.8
La21	1046	15.0	14.4	9.0	5.0	5.0	4.2	4.0
La24	935	10.1	8.8	8.6	6.3	5.1	4.9	5.0
La25	977	11.8	11.6	7.6	5.1	3.9	3.4	2.7
La27	1235	25.0	13.7	11.3	10.2	4.7	4.6	4.1
La29	1152	13.4	14.2	11.0	14.2	8.6	7.4	7.0
La38	1196	15.0	13.3	9.4	9.2	6.9	6.1	5.8
La40	1222	13.3	12.8	9.9	8.7	4.2	4.6	4.1
ABZ7	656	19.7	14.3	11.7	12.5	7.3	6.6	6.7
ABZ8	645	24.7	22.1	17.7	18.5	12.1	11.1	10.9
ABZ9	661	33.4	25.3	22.1	18.0	13.1	11.6	11.2
Mean		17.9	14.2	11.3	9.8	6.4	5.8	5.5

We report this transfer as a property of the method rather than of one rule. Over the 30 independently evolved rules the deterministic pass averages $19.04 \pm 1.97\%$ RE on these instances, individual rules ranging from 15.3% to 23.3%, so the figure attached to any single rule should be read with that spread in mind. The two benchmarks are referred to different bounds, Taillard's for the Taillard-derived instances and best-known ones for the classical set, so their RE values are not directly comparable; what does carry across is the margin over the baselines evaluated on the same instances against the same bound. That margin is preserved: on the classical set the evolved rules average 2.40 times better than MOR and 1.58 times better than G&T-MWKR, against 2.40 and 1.55 on the interval Taillard benchmark they were evolved for.

In terms of quality, the featured rule wrapped in GP-ε at best-of-1024 reaches 11.3% RE against the 9.8% published average of the genetic algorithm, beating it on 3 of the 12 instances; a Wilcoxon signed-rank test on the paired per-instance values puts the difference at the edge of significance ($z = 1.96$, $p \approx 0.05$). The evolved rules therefore do not match the per-instance metaheuristics on this benchmark, and the purpose-built ABC variants remain clearly ahead (5.5–6.4%), as expected of iterative search. What the comparison locates is the constructive frontier: rules evolved once on another benchmark and applied here with no retraining land 1.5 points of RE from a population-based method that searches every instance separately, while the deterministic baselines average 30.1% (G&T-MWKR) and 45.8% (MOR) on the same instances.

7. Analysis

This section analyzes the evolved rules. Section 7.1 examines their composition. The remaining subsections address two questions: whether the evolution driven by the makespan objective extracts measurable value from the width terminals (Section 7.2), and whether an objective that penalizes interval width yields more robust schedules, and at what cost (Section 7.3).

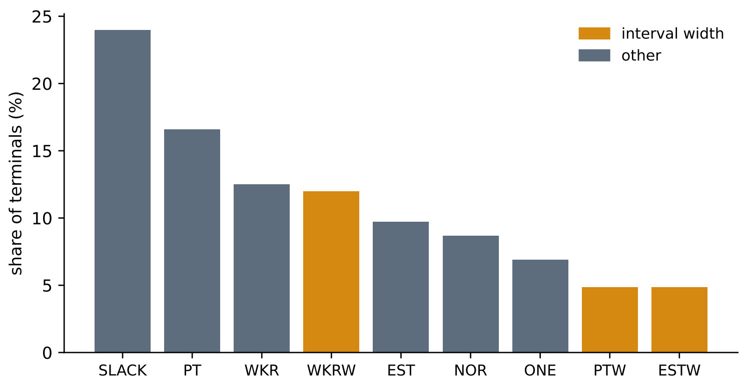

Figure 4: Terminal usage across the 30 evolved rules, as a share of all terminals. Interval-width terminals (amber) expose the magnitude of the uncertainty to the rule.

Figure 5: Sensitivity of the featured rule to the two weights of its simplified form, Eq. (12): mean RE over the 70 instances when each weight is varied and the other is held at its evolved value. Dashed lines mark the evolved weights, the dotted line the mean over the 30 evolved rules. $\beta = 0$ is the rule with its interval-width term switched off.

7.1. Rule anatomy and interpretability

The output of GP is a readable expression. Evolved rules are compact, with a mean tree size of 26 nodes (20–30) and mean depth 9, so a rule can be inspected and simplified by hand. Figure 4 reports how often each terminal is used across the 30 evolved rules. The most frequent are the classical scheduling attributes, slack ($SLACK$), work remaining (WKR) and earliest start (EST), together with worst-case processing time (PT), indicating that the evolution rediscovers known dispatching logic rather than exploiting benchmark artifacts. The *interval-width* terminals (PTW, $ESTW$, $WKRW$) account for ≈21% of all terminal usage and appear in 27 of the 30 evolved rules. Whether that width information translates into better schedules is quantified in Sections 7.2 and 7.3.

The simplified form of the featured rule, Eq. (12), also admits a direct sensitivity analysis: it weighs PT by $\alpha = 2$ and $WKRW$ by $\beta = 1$, and we swept each weight over the 70 instances holding the other at its evolved value (Figure 5). Both curves have a single broad minimum. The evolved α coincides with the minimum of its sweep, indicating that this weight was effectively optimized by the evolution; the evolved β lies near, but not at, the minimum of its own, where $\beta = 0.5$ yields 17.54% against 17.71%. Such a post-hoc adjustment is available for any evolved rule. Setting $\beta = 0$ removes the width term from the deployed rule and costs 0.5 points of RE (18.23 against 17.71).

7.2. The value of interval-awareness: terminal ablation

The interval-width terminals are the distinctive ingredient of this setting, so we isolate their contribution by ablation: we repeat the 30 evolutions with and without the width terminals (PTW, $ESTW$, $WKRW$) in the terminal set, everything else held fixed, and compare the resulting rules with a paired Wilcoxon signed-rank test on the shared seeds. Table 7 collects the outcome.

Under the makespan objective, the width terminals do not improve the

Table 7: Terminal ablation under two fitness objectives (30 independent evolutions per arm; rank-based tests between arms). *Width* is the relative width of the predicted makespan interval. Under both objectives the arm with width terminals produces significantly narrower intervals; the expected makespan does not differ significantly under the makespan objective.

Fitness objective	Terminal set	RE (%)	Width (%)
	full	18.99 ± 1.33	12.41 ± 0.30
Makespan $E[\mathbf{C}_{\max}]$	without widths	18.66 ± 1.08	12.62 ± 0.23
	test	*n.s.* $(z = -0.83)$	$z = 3.40$, $p < 0.001$
	full	19.62 ± 1.52	12.04 ± 0.75
Robust, Eq. (13)	without widths	18.42 ± 0.87	12.47 ± 0.20
	test	—	$z = 3.22$, $p < 0.01$

quantity being optimized. Removing the three of them leaves the expected makespan unchanged: 18.66 ± 1.08 mean RE over the 70 instances without them versus 18.99 ± 1.33 with them, a difference that is not significant ($z = -0.83$) and, if anything, favors the smaller terminal set. They do, however, affect the schedules: the relative width of the predicted makespan interval is significantly narrower when the rule can read widths (12.41 ± 0.30 versus 12.62 ± 0.23; paired Wilcoxon $z = 3.40$, $p < 0.001$), even though nothing in this objective rewards narrowness. The terminals appear in 27 of the 30 rules (Section 7.1); their effect under this objective is a slightly tighter interval at no cost in expected makespan.

The makespan fitness gives the evolution no incentive to trade expected makespan for predictability. To test whether it can make that trade when rewarded for it, we repeat the ablation with a *robust* fitness that explicitly rewards predictability, minimizing

$$f_\lambda = \overline{C}_{\max} + \lambda\,(\overline{C}_{\max} - \underline{C}_{\max}), \tag{13}$$

that is, a low worst case *and* a narrow interval, with the weight λ setting the balance between the two; we take $\lambda = 1$ here. Under this objective the separation widens. Rules evolved with the width terminals produce schedules whose makespan interval is significantly narrower than those evolved without them (12.04 ± 0.75 versus 12.47 ± 0.20 relative width; paired Wilcoxon $z = 3.22$, $p < 0.01$). Without the width terminals the evolution stays at ≈12.5% under both objectives, 12.62 under makespan and 12.47 under the robust fitness, whereas with them it moves from 12.41 to 12.04% as soon as narrowness is rewarded, and further still as λ grows. That movement is paid for in expected makespan (19.62 versus 18.42 RE), which is the trade-off the robust objective asks for.

The weight λ of Eq. (13) governs that trade-off, and sweeping it traces the frontier between quality and predictability (Figure 6). Both quantities move monotonically with λ: as the weight grows from 0.5 to 4, the relative width of the makespan interval falls from 12.18 ± 0.50 to 10.54 ± 1.26 while the expected makespan degrades from 19.20 ± 1.52 to 23.27 ± 3.80 RE. At the far end of the sweep the schedules are therefore narrower than anything the ablated rules can produce, at a cost of about four points of RE. The practitioner can choose an operating point along this curve. No comparable frontier is available to the ablated arm, which stayed at ≈12.5% under both objectives it was evolved for.

Figure 7 shows the four arms rule by rule. Without the width terminals no rule goes below 12.05% under either objective; with them, the minimum is 11.60% under the makespan objective and 9.24% under the robust one.

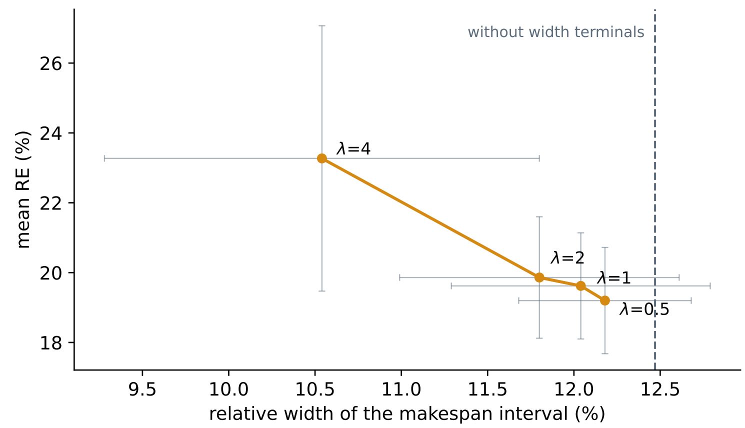

Figure 6: Quality–predictability frontier traced by the weight λ of the robust fitness, Eq. (13). Rules evolved *with* the interval-width terminals; each point is the mean over the independent evolutions of that arm, every rule evaluated on the 70 instances, and the grey bars span ± 1 sd in both coordinates. The dashed line marks the width reached by the arm evolved *without* them under the robust fitness (λ=1). Increasing λ yields narrower makespan intervals at the cost of a larger expected makespan.

Both conditions are therefore necessary: the terminals and an objective that rewards narrowness.

Taken together, these experiments delimit the contribution of interval awareness. The width of an interval is exploitable information for a dispatching rule, but its effect falls on the predictability of the schedule rather than on its expected makespan: exposing the widths leaves the expected makespan unchanged under both objectives while narrowing the predicted interval under both, modestly when the objective disregards narrowness and substantially when it rewards it, with λ governing the extent of the trade-off.

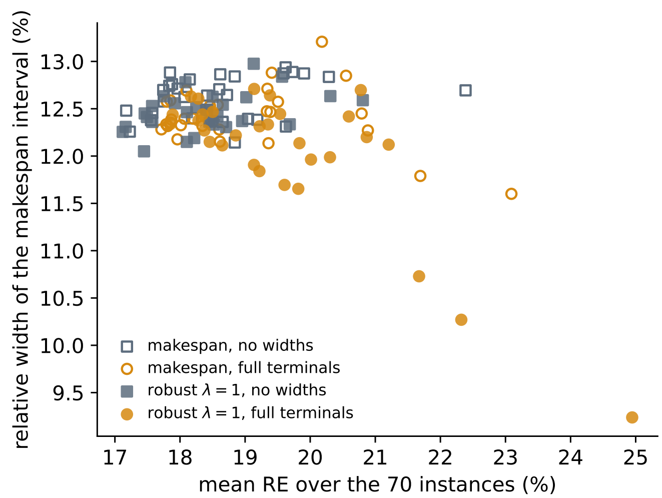

Figure 7: The four ablation arms rule by rule: each point is one evolved rule, evaluated on the 70 instances. The arms without width terminals occupy a band between 12.05 and 12.98% under either objective; only the robust arm with the full terminal set reaches below it, down to 9.24%.

7.3. Robustness under executional uncertainty

Relative error measures a-priori quality; the $\bar{\varepsilon}$ robustness of Eq. (10) measures how well that a-priori prediction survives execution, and is the quantity examined here. We estimate it by Monte Carlo, drawing $K{=}1000$ realizations of the durations uniformly on each operation's interval and executing the fixed processing order on each of them; lower is more robust. To probe behavior under growing uncertainty we also widen every interval symmetrically about its midpoint by +20% and +40% (clipping negative bounds to zero), re-evaluating the same processing orders.

Under the makespan objective the first question receives the same negative answer (Table 8): the featured rule and its ablated counterpart are statistically indistinguishable in $\bar{\varepsilon}$ (5.70 against 5.75 at the nominal width;

Table 8: Predicted quality and executional robustness over the 70 instances (lower is better on all measures): mean RE, relative width of the predicted makespan interval, and $\bar{\varepsilon}$ ($\times 10^3$) at nominal interval widths and widened by +20% and +40% (Monte Carlo, K=1000 realizations, common random numbers across methods). Each GP arm is represented by one rule: the makespan arms by their best RE, the robust arms by their narrowest predicted interval, which is what their fitness optimizes.

			$\bar{\varepsilon}$ ($\times 10^3$)		
Method	RE (%)	Width (%)	+0%	+20%	+40%
GP, makespan	17.71	12.28	5.70	6.64	7.57
GP, makespan, no widths	17.17	12.48	5.75	6.80	7.83
GP, robust λ=1	24.95	9.24	4.52	5.32	6.08
GP, robust λ=1, no widths	17.45	12.05	5.54	6.50	7.42
GP, robust λ=4	28.27	8.61	4.21	5.01	5.73
G&T-MWKR	29.50	13.77	6.29	7.51	8.58
EST	42.30	13.37	5.66	6.79	7.83

paired Wilcoxon over the 70 instances, $z = -0.17$). The rule is more robust than G&T-MWKR (6.29, $z = -4.42$, $p < 0.001$) but not than EST (5.66, $z = 0.42$), although it improves on EST by 24 points of RE. A low deviation does not require a low makespan, so robustness must be optimized for directly.

The robust fitness achieves this. The narrowest rule of the λ=1 arm reduces $\bar{\varepsilon}$ from 5.70 to 4.52 ($z = -6.40$ against the makespan rule), and the reduction requires the width terminals: under the same fitness, its ablated counterpart reaches 5.54 ($z = -6.25$). At λ=4 the deviation falls to 4.21. The narrower predicted intervals of Section 7.2 thus correspond to smaller realized deviations, at the cost in expected makespan reported in the RE

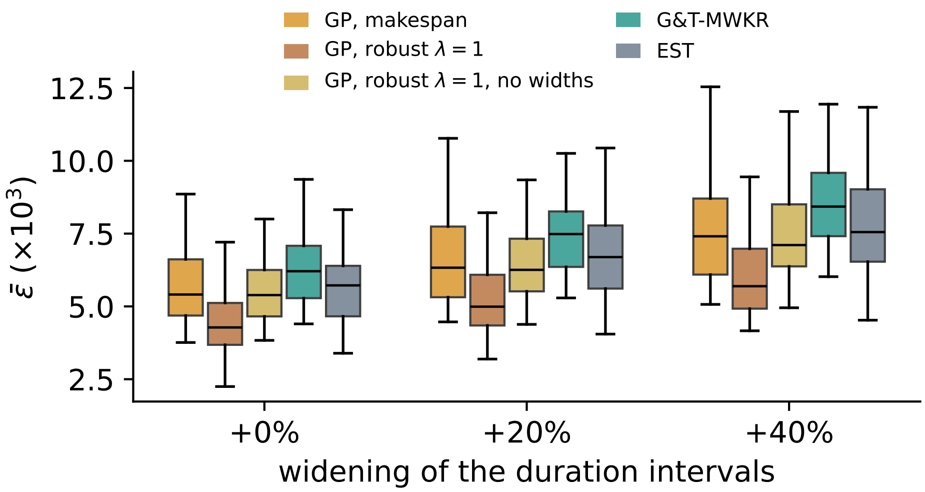

Figure 8: Per-instance $\bar{\varepsilon}$ by method, with the duration intervals at their nominal widths and widened by +20% and +40% (lower is better). The robust λ=1 entries are the narrowest rule of that arm and its counterpart evolved without the width terminals, as in Table 8.

column: 24.95 and 28.27 against 17.71.

Figure 8 shows the per-instance $\bar{\varepsilon}$ distributions. The ordering is stable as uncertainty grows, and the advantage of the robust rule widens with it: at +40% its deviation is 6.08 against 7.57 for the makespan rule and 8.58 for G&T-MWKR.

7.4. Limitations

Three limitations bound the scope of our conclusions. First, all rules operate inside a semi-active SGS; active (insertion-based) generation schemes are known to help weaker constructors, and co-evolving rules with an active SGS in the loop remains untested. Second, all rules were evolved on instances generated with the ±15% symmetric interval scheme standard in the IJSP literature; the robustness analysis of Section 7.3 does re-evaluate the resulting schedules under intervals widened by +20% and +40%, but no rule was evolved under a different width, and asymmetric intervals remain untested. Third, our runtime figures are relative (evolved rules cost the same as hand-crafted ones per pass); absolute times depend on the research-grade Python implementation, but the rankings reported here do not.

8. Conclusions and Future Work

We presented what is, to the best of our knowledge, the first GP hyper-heuristic for job shop scheduling with interval durations, built on a terminal set that exposes interval widths as first-class attributes, with the evolutionary hyperparameters configured automatically by a 200-experiment irace study. The empirical conclusions are three. (i) Evolved rules dominate every deterministic constructive baseline on the 70-instance benchmark by a wide, statistically significant margin (18.99 ± 1.33 over 30 evolutions, 17.71% for the best rule, versus 29.5% for the strongest baseline, which the best rule outperforms on all 70 instances) at the cost of a single constructive pass, and (ii) they generalize zero-shot across all size classes, evolved on 20×15 and best on the unseen 50×15, because the representation contains no size-bound quantities. (iii) Exposing the widths of the uncertain quantities does not improve the expected makespan, since the worst-case bounds already carry that signal; under an objective that penalizes interval width, however, it yields substantially narrower predicted intervals and a deviation of the executed makespan from its prediction about one fifth lower (4.52 against 5.70×10^{-3}), and neither reduction is attainable without the width terminals. The weight λ governs the extent of the trade-off.

Future work follows four lines. First, optimizing the numeric weights of an evolved rule, which the sensitivity analysis of Section 7.1 suggests the evolution leaves unfinished: the weight of *WKRW* in the featured rule is $\beta = 1$, whereas $\beta = 0.5$ scores 17.54% against 17.71%. Two properties of this setting shape how such a search should be posed. Because operations are dispatched by *argmin*, the priority is invariant to adding a constant and to multiplying by a positive one, so the standard technique of linear scaling [28] cannot help and only the *relative* weights of the additive terms are free, $k-1$ of them for a rule with k terms. And because the terminal set contains no ephemeral random constants, the evolution must build coefficients structurally, spending nodes of a bounded budget on arithmetic that a constant would express in one. Both an enriched terminal set and a local search over the relative weights, whether applied after evolution or to the elites of each generation, follow from these observations. Second, richer symbolic models: ensembles of complementary rules [20] and co-evolution of construction and tie-breaking. Third, using evolved rules as inexpensive, high-quality population initializers for the leading IJSP metaheuristics: the pools of diverse, high-quality sequences produced by GP-ε are directly usable as seeded initial populations, and quantifying the effect of such seeding is ongoing work. Fourth, a controlled comparison with learned neural constructive policies under matched training and inference budgets, the head-to-head evaluation that the recent literature [23] identifies as missing.

CRediT authorship contribution statement

Hernán Díaz: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration.

Acknowledgements

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MCIN/AEI/10.13039/501100011033) under grant PID2022-141746OB-I00.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The GP hyper-heuristic is implemented from scratch in pure Python/NumPy, with no evolutionary-computation dependencies; evolved rules are stored as plain JSON expression trees and evaluated as drop-in dispatching strategies by the same environment used by every baseline. The benchmark instances, the generated solutions and rules, and the analysis scripts are shared in a Zenodo repository. [TODO: DOI de Zenodo al depositar]

References

[1] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Fast elitist ABC for makespan optimisation in interval JSP, Natural Computing 22 (4) (2023) 645–657.

[2] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, An elitist seasonal artificial bee colony algorithm for the interval job shop, Integrated Computer-Aided Engineering 30 (3) (2023) 223–242.

[3] J. Branke, S. Nguyen, C. W. Pickardt, M. Zhang, Automated design of production scheduling heuristics: a review, IEEE Transactions on Evolutionary Computation 20 (1) (2016) 110–124.

[4] S. Nguyen, Y. Mei, M. Zhang, Genetic programming for production scheduling: a survey with a unified framework, Complex & Intelligent Systems 3 (1) (2017) 41–66.

[5] J. J. Palacios, I. González-Rodríguez, C. R. Vela, J. Puente, Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop, Fuzzy Sets and Systems 278 (2015) 81–97.

[6] A. Allahverdi, H. Aydilek, A. Aydilek, Single machine scheduling problem with interval processing times to minimize mean weighted completion time, Computers & Operations Research 51 (2014) 200–207.

[7] W. Xu, W. Wu, Y. Wang, Y. He, Z. Lei, Flexible job-shop scheduling method based on interval grey processing time, Applied Intelligence 53 (2023) 14876–14891.

[8] A. Allahverdi, A survey of scheduling problems with uncertain interval/bounded processing/setup times, Journal of Project Management 7 (4) (2022) 255–264.

[9] Y. N. Sotskov, T.-C. Lai, F. Werner, Measures of problem uncertainty for scheduling with interval processing times, OR Spectrum 35 (3) (2013) 659–689.

[10] D. Lei, Population-based neighborhood search for job shop scheduling with interval processing time, Computers & Industrial Engineering 61 (4) (2011) 1200–1208.

[11] H. Díaz, I. González-Rodríguez, J. J. Palacios, I. Díaz, C. R. Vela, A genetic approach to the job shop scheduling problem with interval uncertainty, in: Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU), Springer, 2020, pp. 663–676.

[12] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Robust schedules for tardiness optimization in job shop with interval uncertainty, Logic Journal of the IGPL (2022). `doi:10.1093/jigpal/jzac016`.

[13] S. S. Panwalkar, W. Iskander, A survey of scheduling rules, Operations Research 25 (1) (1977) 45–61.

[14] R. Haupt, A survey of priority rule-based scheduling, OR Spektrum 11 (1) (1989) 3–16.

[15] B. Giffler, G. L. Thompson, Algorithms for solving production-scheduling problems, Operations Research 8 (4) (1960) 487–503.

[16] J. Adams, E. Balas, D. Zawack, The shifting bottleneck procedure for job shop scheduling, Management Science 34 (3) (1988) 391–401.

[17] J. Carlier, The one-machine sequencing problem, European Journal of Operational Research 11 (1) (1982) 42–47.

[18] J. R. Koza, Genetic Programming: On the Programming of Computers by Means of Natural Selection, MIT Press, Cambridge, MA, 1992.

[19] R. Poli, W. B. Langdon, N. F. McPhee, A Field Guide to Genetic Programming, Lulu Enterprises, 2008, freely available at `http://www.gp-field-guide.org.uk`.

[20] F. J. Gil-Gala, C. Mencía, M. R. Sierra, R. Varela, Learning ensembles of priority rules for online scheduling by hybrid evolutionary algorithms, Integrated Computer-Aided Engineering 28 (1) (2021) 65–80.

[21] F. J. Gil-Gala, M. Ðurašević, M. R. Sierra, R. Varela, Tree search hyper-heuristic with application to combinatorial optimization, Journal of Heuristics 31 (2025) 25.

[22] D. Karunakaran, Y. Mei, G. Chen, M. Zhang, Dynamic job shop scheduling under uncertainty using genetic programming, in: Intelligent and Evolutionary Systems, Springer, 2017, pp. 195–210.

[23] M. Xu, Y. Mei, F. Zhang, M. Zhang, Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning, Artificial Intelligence Review 58 (2025) 160.

[24] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, C. Xu, Learning to dispatch for job shop scheduling via deep reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS), 2020.

[25] J. Park, J. Chun, S. H. Kim, Y. Kim, J. Park, Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning, International Journal of Production Research 59 (11) (2021) 3360–3377.

[26] A. Sprecher, R. Kolisch, A. Drexl, Semi-active, active, and non-delay

schedules for the resource-constrained project scheduling problem, European Journal of Operational Research 80 (1) (1995) 94–102.

[27] F. J. Gil-Gala, M. Đurašević, M. R. Sierra, J. Puente, Greedy randomised adaptive solution builder using priority rules, in: Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '25), ACM, 2025, pp. 211–214.

[28] M. Keijzer, Improving symbolic regression with interval arithmetic and linear scaling, in: Genetic Programming (EuroGP), Springer, 2003, pp. 70–82.

Appendix A. Per-instance results

Table A.9 lists, for each of the 70 interval Taillard instances, the crisp lower bound and the single-pass RE (%) of the evolved rule and of the two strongest baselines: EST, the best of the plain dispatching rules, and G&T-MWKR, the best baseline overall.

Table A.9: Per-instance RE (%) of the evolved rule (GP) and of the two strongest baselines, EST and G&T-MWKR (single deterministic pass). LB is the crisp Taillard lower bound.

Inst.	LB	EST	G&T	GP	Inst.	LB	EST	G&T	GP
TA1	1231	49.9	26.1	19.3	TA36	1819	41.1	42.8	23.8
TA2	1244	24.9	27.8	14.2	TA37	1771	43.4	35.6	18.4
TA3	1218	33.0	33.2	14.1	TA38	1673	45.0	28.0	20.1
TA4	1175	32.0	27.7	13.0	TA39	1795	46.6	28.9	22.4
TA5	1224	27.2	33.0	14.7	TA40	1651	46.9	42.3	26.1
TA6	1238	26.8	18.2	13.7	TA41	1906	44.6	45.0	33.4
TA7	1227	30.7	26.1	17.7	TA42	1884	71.1	38.1	24.3
TA8	1217	23.7	25.0	14.8	TA43	1809	68.7	39.8	22.3
TA9	1274	37.7	28.2	20.0	TA44	1948	52.1	34.7	25.8
TA10	1241	37.0	22.1	15.4	TA45	1997	50.2	33.7	14.9
TA11	1357	56.2	30.7	17.5	TA46	1957	60.8	31.6	22.4
TA12	1367	48.8	34.6	17.6	TA47	1807	46.8	30.1	25.6
TA13	1342	60.2	26.9	13.3	TA48	1912	59.1	33.7	21.1
TA14	1345	39.5	29.7	13.2	TA49	1931	51.0	35.3	20.1
TA15	1339	48.1	30.6	16.7	TA50	1833	61.4	46.2	31.3
TA16	1360	45.2	34.0	15.6	TA51	2760	33.8	34.7	23.2
TA17	1462	58.9	17.7	13.9	TA52	2756	38.0	26.5	12.7
TA18	1377	43.8	31.8	23.6	TA53	2717	34.1	18.3	11.8
TA19	1332	45.8	29.3	20.0	TA54	2839	20.9	16.1	8.2
TA20	1348	45.7	24.1	15.1	TA55	2679	31.7	29.2	15.5
TA21	1642	33.4	28.0	20.2	TA56	2781	34.8	19.8	10.9
TA22	1561	44.2	30.8	17.4	TA57	2943	32.9	20.0	9.6
TA23	1518	42.8	35.3	17.8	TA58	2885	37.4	26.3	13.6
TA24	1644	56.3	23.8	14.1	TA59	2655	26.6	28.2	16.7
TA25	1558	45.8	33.8	16.6	TA60	2723	41.7	19.8	15.5
TA26	1591	42.8	31.2	26.9	TA61	2868	45.7	25.4	15.4
TA27	1652	50.0	31.1	21.4	TA62	2869	43.0	27.6	17.2
TA28	1603	36.8	26.8	12.4	TA63	2755	38.8	23.8	13.4
TA29	1583	28.8	32.0	17.2	TA64	2702	38.2	28.4	12.2
TA30	1528	34.2	36.1	17.0	TA65	2725	42.9	30.0	20.5
TA31	1764	30.7	34.4	24.5	TA66	2845	41.8	28.0	11.9
TA32	1774	42.7	33.3	19.0	TA67	2825	31.7	26.1	15.2
TA33	1788	51.3	34.8	27.5	TA68	2784	50.1	19.1	10.6
TA34	1828	45.9	31.8	20.7	TA69	3071	35.3	22.8	12.9
TA35	2007	30.3	24.1	8.4	TA70	2995	42.9	23.4	16.7